

Support de cours

Kubernetes

Fondamentaux

HI-TECH ACADEMY · 7 h – 1 journée · 100 % à distance

Version 1.0 · dernière mise à jour : 21/06/2026

Sommaire de la journée

1

MATIN (3 H 30)

Découvrir Kubernetes et déployer ses premiers pods

Orchestration, architecture du cluster, Pods, kubectl — TP 1 : premier déploiement sur AKS.

2

APRÈS-MIDI (3 H 30)

Déployer, exposer et configurer une application

Deployments, Services (survol d'Ingress), ConfigMap & Secrets — TP 2 fil rouge + évaluation finale.

1

MATIN (3 H 30)

**Découvrir Kubernetes et déployer
ses premiers pods**

À quoi sert Kubernetes ?

Kubernetes (K8s) est un orchestrateur de conteneurs : il automatise le déploiement, la mise à l'échelle et la résilience des applications conteneurisées.

Déploiement

Déployer et mettre à jour sans interruption.

Résilience

Redémarrer / remplacer les conteneurs défectueux.

Mise à l'échelle

Adapter le nombre de réplicas à la charge.

Abstraction

Masquer la complexité de l'infrastructure.

Conteneurs, puis orchestration

- Un conteneur empaquète une application et ses dépendances.
- À l'échelle, tout gérer à la main devient impossible.
- Kubernetes orchestre : placement, réseau, montée en charge, auto-réparation.
- Écosystème porté par la CNCF (Cloud Native Computing Foundation).

Cas d'usage

Microservices, API, sites web, traitements par lots, plateformes internes.

Bénéfice clé

Portabilité entre environnements (on-premise, Azure, autres clouds).

Architecture d'un cluster



Le control plane décide ; les nœuds exécutent les pods. kubectl dialogue avec le kube-apiserver.

Composants du control plane

kube-apiserver

Point d'entrée unique : reçoit toutes les requêtes (kubectl, contrôleurs).

etcd

Base clé-valeur qui stocke l'état complet du cluster.

kube-scheduler

Décide sur quel nœud placer chaque nouveau pod.

controller-manager

Exécute les boucles de contrôle (réplicas, nœuds, etc.).

Boucle de réconciliation

1 · État désiré

Décrit dans vos manifestes YAML.

2 · Observation

Kubernetes compare désiré et réel.

3 · Action

Crée, supprime ou remplace pour converger.

Le contrôleur boucle en continu pour maintenir l'état désiré : c'est l'auto-réparation.

Prise en main : kubectl

kubectl est l'outil en ligne de commande pour piloter le cluster.

COMMANDES ESSENTIELLES

```
kubectl get nodes           # lister les noeuds
kubectl get pods -A         # tous les pods, tous les namespaces
kubectl apply -f fichier.yaml # creer / mettre a jour
kubectl describe pod web    # details et evenements
kubectl logs web            # logs d'un pod
kubectl exec -it web -- sh  # shell dans un pod
```

Votre premier Pod

POD.YAML

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
    - name: nginx
      image: nginx:1.27
      ports:
        - containerPort: 80
```

- Un Pod est la plus petite unité déployable.
- Il encapsule un (ou plusieurs) conteneur(s).
- On l'applique avec `kubectl apply -f pod.yaml`.
- En pratique, on déploie via un Deployment, pas un Pod nu.

TP 1 – Premiers pas sur le cluster AKS

- 1 Se connecter au cluster AKS fourni et à son espace de noms dédié.
- 2 Déployer un premier Pod nginx à partir du manifeste fourni.
- 3 Inspecter les ressources : `kubectl get`, `describe`, `logs`, `exec`.
- 4 Nettoyer : supprimer le Pod et vérifier la disparition des ressources.

Corrigé commenté en direct — les manifestes sont fournis sur l'espace de la formation.

2

APRÈS-MIDI (3 H 30)

**Déployer, exposer et configurer
une application**

Pod, ReplicaSet, Deployment

Deployment

Gère les versions et les mises à jour

ReplicaSet

Maintient le nombre de réplicas

Pods

Exécutent réellement les conteneurs

À retenir

On déclare un Deployment ; il crée un ReplicaSet, qui crée les Pods. La hiérarchie assure résilience et mises à jour maîtrisées.

Un Deployment en YAML

DEPLOYMENT.YAML

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 3
  selector:
    matchLabels: { app: web }
  template:
    metadata:
      labels: { app: web }
    spec:
      containers:
        - name: web
          image: nginx:1.27
```

- replicas : nombre de pods souhaités.
- selector / labels : lien Deployment → Pods.
- template : le modèle de pod déployé.
- Modifier l'image déclenche une mise à jour.

Rolling update & rollback

- Mise à jour progressive : les pods sont remplacés par lots, sans coupure.
- Kubernetes surveille la santé avant de poursuivre.
- En cas de problème : retour arrière immédiat.

MISE À JOUR / ROLLBACK

```
kubectl set image \
  deploy/web web=nginx:1.28
kubectl rollout status deploy/web
kubectl rollout undo deploy/web
```

Les Services

ClusterIP

Accès interne au cluster uniquement (par défaut).

NodePort

Ouvre un port sur chaque nœud du cluster.

LoadBalancer

Expose via un load balancer externe (cloud).

Un Service fournit une adresse stable et répartit le trafic vers les pods correspondant au selector.

Service en YAML · survol d'Ingress

SERVICE.YAML

```
apiVersion: v1
kind: Service
metadata:
  name: web
spec:
  type: ClusterIP
  selector: { app: web }
  ports:
    - port: 80
      targetPort: 80
```

Et pour l'HTTP public ? Ingress (survol)

- Routage HTTP par nom de domaine et par chemin.
- Un seul point d'entrée pour plusieurs services.
- Nécessite un Ingress Controller (ex. NGINX).
- Approfondi dans le module avancé.

ConfigMap & Secrets

CONFIGMAP.YAML

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_MODE: "production"
  LOG_LEVEL: "info"
```

SECRET.YAML

```
apiVersion: v1
kind: Secret
metadata:
  name: app-secret
type: Opaque
data:
  password: cGFzc3dvcmQ= # base64
```

ConfigMap : configuration non sensible. **Secret** : données sensibles (base64, à protéger par RBAC).

TP 2 – Fil rouge, puis évaluation finale

Sur le cluster Azure (AKS) fourni, déployez une application complète de bout en bout :

- 1 Déployer via un Deployment (au moins 2 réplicas).
- 2 Exposer l'application avec un Service.
- 3 Externaliser la configuration (ConfigMap + Secret).
- 4 Vérifier l'état de l'application avec kubectl (get, describe, logs).

La journée se conclut par l'évaluation finale (QCM + mise en pratique), le bilan et la correction commentée.

Pour aller plus loin

Module avancé

DaemonSet et StatefulSet, stockage persistant (PV/PVC), placement des pods (affinity / anti-affinity, taints), Ingress approfondi, opérateurs, GitOps.

Ressources

Documentation officielle Kubernetes, CNCF, environnements de lab, certifications CKA / CKAD.

Vous savez désormais déployer, exposer et configurer des applications sur Kubernetes.

Merci !

Des questions ? Contactez votre formateur.